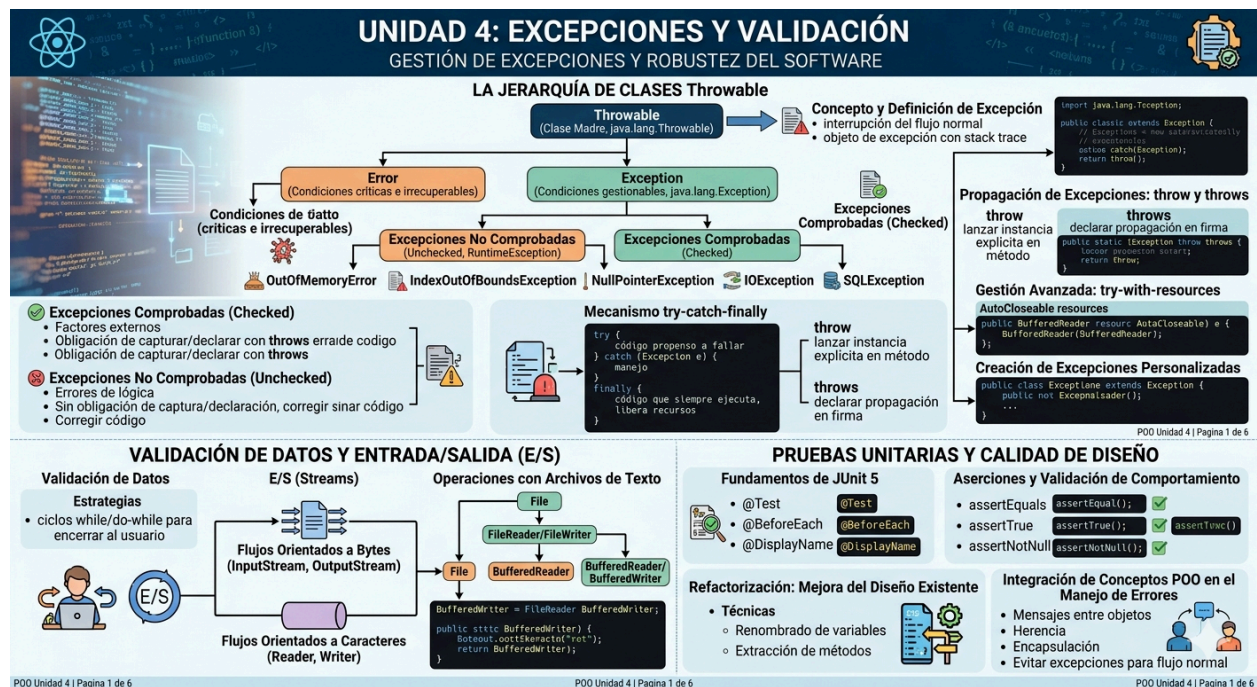




Programación Orientada a Objetos



Mignani Agustina

Universidad Provincial de Córdoba

Programación Full Stack - 2 año

Guerra Mario

15 de junio, 2026

Detalle de los Cambios Realizados en el Proyecto

1. Creación de una Excepción Personalizada (Checked Exception)

Para modelar de forma semántica el problema de negocio que ocurre cuando un curso se queda sin vacantes, creé una nueva clase independiente en el sistema llamada `CupoExcedidoException.java`.

Cómo decidí que herede de forma directa de la superclase `java.lang.Exception`, la definí como una excepción comprobada (checked). Al usar una checked, mi código ahora obliga explícitamente a cualquier método que invoque la inscripción a manejar el escenario de error. De esta manera, me aseguré de que la aplicación no ignore esta situación y reaccione de forma controlada.

2. Lanzamiento y Propagación de Errores en la Clase Curso

Eliminé por completo la lógica condicional que solo imprimía mensajes fijos en la consola ante un fallo y la reemplacé por un sistema real de detección de errores dentro del método `procesarInscripcion(Alumno alumno)`:

Palabra clave throw: Si la lista de inscritos alcanza el atributo `cupoMax`, apliqué un `throw new CupoExcedidoException(...)` para interrumpir el flujo normal de las instrucciones lanzando el error de forma explícita con un mensaje dinámico.

Palabra clave throws: Como es una excepción comprobada y no la atajo adentro de la misma clase, modifiqué la firma del método y sus dos versiones sobrecargadas para anunciar y propagar la excepción hacia el método que la llamó originalmente (`Main.java`).

3. Validaciones Robustas en Constructores y Setters (Unchecked Exceptions)

Para cumplir con el principio de encapsulación, incorporé validaciones defensivas usando la excepción estándar no comprobada `IllegalArgumentException` (que deriva de `RuntimeException`):

En la clase Curso: Modifiqué el constructor y el setter `setCupoMax(int cupoMax)` para lanzar un error inmediato si se intenta pasar un cupo menor o igual a cero, protegiendo así el estado interno del objeto para que no admita datos incoherentes. También validé en el constructor que el objeto `Profesor` no sea nulo.

En la clase Calificación: Refactoricé el método `setNota(double nota)` para que, si el valor recibido no está entre 0 y 10, lance la excepción. Así logré frenar la asignación de datos corruptos apenas se detectan.

4. Implementación del Mecanismo try-catch en la Clase de Control (Main)

Para evitar que el programa sufra una finalización bruta, adecuó la clase Main.java:

Lo que hice fue meter las llamadas críticas de inscripción adentro de bloques try para cubrirlas y dejarlas bajo vigilancia por si saltaba el problema del cupo.

Definí bloques catch específicos para capturar de forma discriminada tanto a CupoExcedidoException como a IllegalArgumentException. Esto me permitió atrapar los errores y mostrar los mensajes de forma limpia, haciendo que el hilo del programa continúe corriendo con total normalidad.

5. Calidad de Diseño y Refactorización Visual

Aplicó técnicas fundamentales de refactorización sobre el entorno de prueba del Main:

Eliminación de Acoplamiento Rígido: Saqué todas las cadenas de texto fijas con nombres propios de los mensajes de la consola en las fases de prueba 2, 3 y 4.

Uso de Métodos Observadores: Vinculé los títulos y alertas del sistema de manera dinámica utilizando las variables del entorno mediante llamadas como alumnoAgustina.getNombreCompleto() o cursoPOO.getNombre().toUpperCase().

Resultado: Conseguí que el diseño del código sea semántico y legible. Ahora, cualquier cambio que suceda una instancia se actualiza solo en la terminal, garantizando un código mucho más mantenible y profesional.